# Ders 20 Çalışma Özeti

## Ders 20 Çalışma Özeti

### Genel Konular

- Ders üç bölümden oluşur:
  - Doktor-teşhis sorusunun kalan kısımlarının çözümü.
  - Özyinelemeli (recursive) fonksiyonlar: faktöriyel, üs alma, hızlı üs alma, recursion tree ile karmaşıklık analizi, merge sort ve quick sort'un $n \log n$ karmaşıklığı ve diğer grubun laboratuvarındaki özyineleme soruları.
  - Final projesinin ayrıntıları.
- Doktor-teşhis sorusu (önceki dersin devamı)
  - Hastalık kodları (1-100) kategoriktir. 100. ve 80. hastalık arasında fark almak anlamsızdır. Yalnızca eşit olup olmadıklarına bakılır.
  - En başarılı doktor: her doktorun satırı doğru teşhis dizisiyle karşılaştırılır, `D[i][j] == dogru[j]` ise sayaç artar. Sayaçlar bir skor dizisine yazılır ve bu dizinin en büyüğü bulunur.
  - Birbirine en çok benzeyen iki doktor: `i` 0'dan `D-1`'e, `j` `i+1`'den başlar, `k` hastalar üzerinde döner. `D[i][k] == D[j][k]` ise benzerlik artar. Bütün benzerlikleri saklamaya gerek yoktur. En iyi değer ve indisleri tek değişkenlerde tutulur, daha iyisi bulununca güncellenir.
  - Hocanın gösterdiği hazır çözümde önce matrisin transpozu alınmış, sonra kategorik `findDistance` ile en yakın çift aranmış. Hoca transpoza gerek olmadığını söyler. C'de bir fonksiyon matris döndüremez, matris parametre olarak gönderilip içi doldurulur.
  - "Demokrasi usulü" teşhis (kimler mağdur olur?): her hasta için doktorların koyduğu teşhislerin frekansı (histogram) bulunur. En sık konan teşhis en büyüğü bulma fonksiyonuyla seçilir ve doğru teşhisle karşılaştırılır. Farklıysa o hasta mağdurdur ve bir dizide işaretlenir.
  - Hoca, frekans bulma, en büyüğü bulma gibi fonksiyonların genel amaçlı olduğunu ve başka senaryolarda da kullanılabileceğini vurgular. Gelecek dönem bunlar ayrı bir C dosyasına konulup her programda kullanılacak. Nesneye yönelik programlamada da tipten bağımsız yazılacak.
- Özyinelemeye dönüş: faktöriyel
  - Her fonksiyon çağrısında dönüş için gereken bilgiler stack'te saklanır. Özyinelemeli fonksiyon da normal bir fonksiyon çağrısıdır, yalnızca kendini çağırır. Çıkış (base case) yazılmazsa sonsuza kadar çağırır.
  - Base case matematikten gelir: $0! = 1$ ve negatif sayıların faktöriyeli tanımsızdır.
  - Bağıntı: $f(n) = 1$ ($n = 0$), $f(n) = n \cdot f(n-1)$ ($n > 0$). Bu bir azalt ve yönet (decrease and conquer) yöntemidir.
  - Kod: önce `if (n < 0) return -1;` (hatalı giriş), sonra `if (n == 0) return 1;`, değilse `return n * fakt(n - 1);`. `main`'de sonuç -1 ise "Sonuç hatalı" yazdırılır. `fakt(3)` 6 verdi, `fakt(-3)` hatalı sonuç verdi.
  - Her durumun base case'i problemin matematiğine göre değişir.
  - Stack ile izleme: `fakt(3)`, `3 * f(2)`, `2 * f(1)`, `1 * f(0)` sırasıyla stack'e eklenir. `f(0)` 1 döndürür. Sonra en son çağrılan ilk tamamlanır (LIFO, iç içe parantezler gibi): $1 \cdot 1 = 1$, $2 \cdot 1 = 2$, $3 \cdot 2 = 6$. Özyinelemeli kodları adım adım takip etmenin yolu stack çizmektir. `main` de stack'tedir.
  - Bir öğrenci `return` ile `break`'i karıştırdı. Hoca bunların farklı olduğunu söyler.
- Faktöriyel analizi
  - İteratif çözüm $n$ çarpma yapar. Özyinelemeli çözüm de $n$ kez çağrılır ve $n$ çarpma

yapar. Üstelik ek stack kullandığı için daha kötüdür.
  - Özyinelemenin asıl faydası böl ve yönet (divide and conquer) algoritmalarında görülür. İteratif sıralamalar (selection, insertion, bubble) $n^2$, merge sort ve quick sort $n \log n$'dir. Counting sort gibi $n$'lik yöntemler genel amaçlı sıralama olarak kullanılamaz.
- Üs alma (`p(x, n)`)
  - $x^0 = 1$ ($x \neq 0$), $x^1 = x$, $x^n = x \cdot x^{n-1}$. Negatif üs için $x^{-n} = \left(\frac{1}{x}\right)^n$. Parçalı fonksiyonun dört durumu vardır.
  - Örnek: `p(2, 3)`, `2 * p(2, 2)`, `2 * p(2, 1)`, base case 2 döndürür. Sonuç $2 \cdot 2 \cdot 2 = 8$. `p(2, -3)`, `p(0.5, 3)`'e dönüşür ve $\frac{1}{8}$ bulunur.
  - Kodda `1/x` tam sayı bölmesi olmasın diye `1.0/x` yazılır ve fonksiyon `double` döndürür. Ekrana `%lf` ile yazılır.
  - Özel durumlar (`n == 0`, `n == 1`) en başa yazılmalıdır, yoksa alttaki çağrılar işleri karıştırabilir.
  - İteratif sürüm: `r = 1; for (i = 0; i < n; i++) r = r * x;`. İki sürüm de $n$ çarpma yapar.
  - Çıktı: `p(2, 3) = 8`, `p(2, 0) = 1`, `p(2, -3) = 0.125`.
  - Hocanın gösterdiği diğer yazımlar: negatif üs için yalnızca negatif durumda çalışan `1.0 / x * p(x, n+1)` biçimi ve bütün durumları kapsayan, negatif üste sonucu en sonunda `1.0`'a bölen üçüncü bir sürüm.
- Fatih Hoca'nın ders notundaki sorular
  - `T = f(a, b)`: `b == 0` ise `T = a`, değilse `T = f(a + 1, b - 1)`. `f(12, 4)` 16 verir. Yani $b > 0$ için $a + b$'yi hesaplar. Tek bir toplamayla sabit zamanda yapılabilecek işi $b$ adımda yapar.
  - `T = f(dizi, b)`: `b == 1` ise `dizi(b)`, değilse `dizi(b) + f(dizi, b - 1)`. Dizinin elemanlarını sondan başa toplar. $n$ adım ve ek stack belleği kullanır.
  - Özyinelemeli sıralı arama: `b < 1` ise -1 (bulunamadı), `dizi(b) == x` ise `b`, değilse `b - 1` için tekrar çağır. En kötü durum yine $n$'dir.
  - Bu üç örnekte özyineleme zamanı iyileştirmez, bellek açısından daha kötüdür.
- Recursion tree (özyineleme ağacı)
  - Özyinelemeli algoritmanın derinliğini ve toplam işlem sayısını bulmak için kullanılır, özellikle böl ve yönet algoritmalarında.
  - Faktöriyel: her düğüm 1 çarpma yapar ve bir alt düğüm çağırır ($f(n), f(n-1), \ldots, f(0)$). Derinlik $n$, toplam işlem $n$.
  - Fibonacci ($f(n-1) + f(n-2)$) için ağaç yöntemi pek işe yaramaz. Başka yöntemler ayrık matematikte görülecek.
- Hızlı üs alma (`power_v2`)
  - $x^n = x^{n/2} \cdot x^{n/2}$ ($n$ çift), $x^n = x^{\lfloor n/2 \rfloor} \cdot x^{\lfloor n/2 \rfloor} \cdot x$ ($n$ tek). Örnek: $x^7 = x^3 \cdot x^3 \cdot x$. $x^3$ bir kez hesaplanırsa tekrar hesaplamaya gerek kalmaz.
  - Kod: `if (y == 0) return 1; temp = power_v2(x, y / 2); if (y % 2 == 0) return temp * temp; else return temp * temp * x;`. `y` tek de olsa çift de olsa `y / 2` aynı alt problemi verir ($x^5$ ve $x^4$ için $x^2$).
  - `temp` değişkeni örtüşen alt problemleri (overlapping subproblems) bir kez hesaplamayı sağlar. Böl ve yönetin dezavantajı aynı işi tekrar tekrar yaptırmasıdır, ara değişken bunu önler.
  - Analiz: her çağrıda sabit sayıda işlem (bir atama, bir çarpma, bir kontrol) var ve her çağrı tek bir alt çağrı yapar: $n, n/2, n/4, \ldots, 1$. $\frac{n}{2^i} = 1$ olduğunda $i = \log_2 n$. Karmaşıklık $\log n$'dir. $n$'den $\log n$'e iniş çok büyük bir iyileştirmedir.
  - Sabit zaman (constant) 1, doğrusal $n$, karesel $n^2$, logaritmik $\log n$, doğrusal-logaritmik $n \log n$ olarak gösterilir. Sabit sayıdaki işlemler (2, 5) yine 1 kabul edilir.
  - İteratif bir alternatif üssün ikili açılımını kullanır: $x^{10}$ için $10 = 1010_2$. Her bitte kare alınır, bit 1 ise ayrıca $x$ ile çarpılır (dönem başında görülen yöntem).
- `temp` olmadan hızlı üs alma (`power_v3`)
  - `return power_v3(x, y / 2) * power_v3(x, y / 2);` yazılırsa aynı alt problem iki kez hesaplanır ve ağaç her düzeyde ikiye dallanır.
  - Düzeylerde $2^0, 2^1, \ldots, 2^h$ düğüm vardır ve $h = \log_2 n$. Toplam $\sum_{i=0}^{\log_2 n} 2^i = 2^{\log_2 n + 1} - 1 = 2n - 1$. Karmaşıklık tekrar $n$'e çıkar. Derste önce yanlışlıkla $\log n + \log n$ dendi,

ağaç yeniden çizilerek düzeltildi.
- Merge sort ve quick sort
  - Merge sort: dizi tek elemanlı alt dizilere kalana kadar ikiye bölünür (8 6 3 5, sonra 8 6 ve 3 5, sonra tek elemanlar). Sonra iki sıralı dizi birleştirme (merge) ile birleştirilir (6 8, 3 5, ardından 3 5 6 8). İki sıralı diziyi birleştirmek $n$ adımdır.
  - Bağıntı: $T(n) = 2T(n/2) + n$. Ağaçta her düzeyin toplam maliyeti $n$'dir ($2^i$ tane $n/2^i$). Derinlik $\log_2 n$. Toplam $n \log n$.
  - Merge sort ek bellek kullanır. Quick sort ek dizi kullanmadan $n \log n$'de sıralar. Bunların kodlaması Veri Yapıları ve Algoritmalar dersinin konusudur.
- Diğer grubun laboratuvarındaki özyineleme soruları (kodlar paylaşılacak)
  - String uzunluğu: pointer her çağrıda bir ilerletilir, `'\0'` karakterine gelince durulur, her çağrıda 1 eklenir. "Ahmet" için 5.
  - Basamak sayısı: sayı 0'dan büyükse 1 ekle ve sayıyı 10'a bölerek tekrar çağır. 123 için 3.
  - 1'den $n$'e toplam: $n + f(n-1)$.
  - OBEB: Öklid algoritmasıyla $\gcd(16,6) = \gcd(6,4) = \gcd(4,2) = \gcd(2,0) = 2$. Laboratuvar çözümünde bölme yerine çıkarma kullanılmış: `a != b` iken büyük olandan küçüğü çıkarıp tekrar çağırmak.
  - String'i tersten yazdırma: önce bir sonraki karakter için kendini çağırır, `'\0'`'a gelince geri dönerken karakterleri yazar. "yaz" için "zay".
- Final projesi
  - Derste ve sayısal analiz dersinde görülmemiş, bu dersin kapsamı dışında kalan bir algoritma seçilir. Derste anlatılana benzer algoritmalar (ör. selection sort anlatıldı diye başka bir sıralama, quick sort, merge sort) kabul edilmez. Hoca heap sort'un belki olabileceğini ekler.
  - Algoritma C'de, modüler (fonksiyonlarla) yazılır. Global değişken, `goto`, `switch` dışında `break` kullanılmaz.
  - Çalışma zamanı ve bellek gereksiniminden en az biri kod içinde otomatik ölçülür. Farklı girdi boyutları (100, 1000, 10.000 eleman gibi) için süre, adım sayısı veya bellek gösterilir. Bu analiz yoksa ödev geçersiz sayılır.
  - Karşılaştırma, girdi boyutuna göre performansın nasıl değiştiğidir. İstenirse çok bilinen bir algoritmayla da karşılaştırılabilir (ör. doğrusal arama ile ikili arama).
  - 5-7 dakikalık, sesli bir video hazırlanır (YouTube veya Google Drive). Videoda algoritma tanıtılır, kod anlatılır, analiz sonuçları açıklanır. Videonun kendisi değil linki teslim edilir.
  - Raporda algoritmanın tarifi, çalışma prensibi, kullanıldığı alanlar, kısıtları ve karmaşıklığı yer alır. Teslim edilenler: C kodu, rapor, video linki.
  - Hocanın tahminine göre iş yükü 5-6 saat, en fazla bir-iki gündür.

## Hocanın Özellikle Vurguladığı Kısımlar

- Özyinelemeli fonksiyonun iki aşaması: bağıntıyı yazmak ve base case'i belirlemek. Hatalı girişleri ve özel durumları fonksiyonun en başında kontrol edin.
- Özyinelemeli kodları stack çizerek adım adım takip edin.
- Özyineleme her zaman daha hızlı değildir. Faktöriyelde ve basit üs almada iteratif sürümden kötüdür. Faydası böl ve yönet algoritmalarında görülür.
- Aynı alt problemi iki kez çağırmayın, sonucunu bir değişkende saklayın. Aksi halde $\log n$ olan algoritma $n$'e çıkar.
- Projede hazır kod bulunabilir ama analiz, rapor ve video sizin olmalı. Kopya kontrolü yapılacak.
- Derste yapılan örneklerin kodlarını yazıp adım adım inceleyin.

## Kısa Tekrar Notları

- `fakt(n)`: `n == 0` ise 1, değilse `n * fakt(n - 1)`. $n$ çarpma.
- `p(x, n)`: $n < 0$ ise `p(1.0 / x, -n)`, $n = 1$ ise `x`, değilse `x * p(x, n - 1)`. $n$ çarpma.
- Hızlı üs: `temp = p(x, n / 2)`, çiftse `temp * temp`, tekse `temp * temp * x`. $\log n$.
- `temp` yoksa iki çağrı yapılır: $2n - 1$ düğüm, karmaşıklık $n$.
- Merge sort: $T(n) = 2T(n/2) + n$, $n \log n$.

## Detaylı Açıklamalar

### Üs alma sürümlerinin denenmesi

Derste yazılan fonksiyonlar `double` ile bir test programında toplandı ve çağrı sayıları bir sayaçla ölçüldü:

```
double pv2(double x, int y){        /* temp ile */
    double temp;
    if (y == 0) return 1;
    temp = pv2(x, y / 2);
    if (y % 2 == 0) return temp * temp;
    return temp * temp * x;
}
double pv3(double x, int y){        /* temp olmadan */
    if (y == 0) return 1;
    if (y % 2 == 0) return pv3(x, y / 2) * pv3(x, y / 2);
    return pv3(x, y / 2) * pv3(x, y / 2) * x;
}
```

İkisi de $2^{10} = 1024$ sonucunu verdi. $n = 1024$ için `pv2` 12 kez, `pv3` 4095 kez çağrıldı. İlki $\log_2 n$ mertebesindedir (`y` 1024, 512, ..., 1, 0). İkincisi yaklaşık $4n$ çağrı yapar, yani doğrusaldır.

### Ders sonrası notlar

- **`return` olmadan özyinelemeli çağrı:** Ekrandaki kodda negatif üs dalı `p(1.0/X, -n);` olarak yazılmıştı, başında `return` yoktu. Hoca bunun bilerek yapıldığını ve `return` yazılırsa fazladan bir çağrı olacağını söyledi. Bu doğru değildir. `return` olmadan iç çağrının sonucu kullanılmadan atılır ve fonksiyon değer döndürmeden biter. Bu tanımsız davranıştır ve derleyici `-Wall` ile "control reaches end of non-void function" uyarısı verir. Derste ve ders sonrasındaki denemede 0.125 sonucu çıktı, çünkü x86'da iç çağrının sonucu tesadüfen dönüş yazmacında kalıyor. Başka bir derleyicide veya ayarda yanlış sonuç alınabilir. Doğrusu `return p(1.0 / x, -n);`'dir. Ek çağrı sayısı aynıdır.
- **Sayaçların neden yanlış çıktığı:** Derste çağrı sayısını görmek için `printf("%lf--%d", power_v2(2, 10), c);` gibi tek bir `printf` kullanıldı. Ekranda 9 ve 1 gibi beklenmeyen değerler çıktı. C'de bir fonksiyonun argümanlarının hangi sırayla hesaplanacağı belirsizdir (unspecified). GCC çoğunlukla sağdan sola hesaplar, yani sayaç fonksiyon çağrılmadan önce okunur. Ders sonrasındaki denemede de aynı yazımla sayaç 0 çıktı. Doğru ölçüm için önce fonksiyon çağrılıp sonuç bir değişkene atanmalı, sayaç ayrı bir satırda yazdırılmalıdır.
- **Quick sort'un karmaşıklığı:** Quick sort ortalama durumda $n \log n$'dir. Pivot hep en küçük veya en büyük eleman seçilirse (ör. sıralı diziye ilk elemanı pivot almak) en kötü durumda $n^2$'ye çıkar. Merge sort her durumda $n \log n$'dir ama ek bellek kullanır.
- **`temp`'li hızlı üs alma ve çağrı sayısı:** $\log_2 n$ derinlik doğrudur. Tam çağrı sayısı $\lfloor \log_2 n \rfloor + 2$'dir ($y = 0$'a kadar inildiği için). Mertebe değişmez.
- **Özyineleme ve sıralama:** $n \log n$ sıralamaları mümkün kılan özyinelemenin kendisi değil, problemi eşit parçalara bölme fikridir. Merge sort iteratif olarak da (aşağıdan yukarıya, bottom-up) yazılabilir ve yine $n \log n$'dir.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.